

Национальный исследовательский университет ИТМО
Мегафакультет компьютерных технологий и управления
Факультет программной инженерии и компьютерной техники

Лабораторная работа №1

по курсу «Механизмы надежности и безопасности вычислительных систем»

Тема: «Темпоральные выражения»

Выполнили студенты группы Р4219

(подпись)

Симоновский Д.Л.

Руководитель

(подпись)

Пинкевич В.Ю.

16 сентября 2026 г.

Санкт-Петербург

2026

Оглавление

1. Задача	2
2. Ход работы	2
2.1. LTL формулы.....	2
2.2. Инфраструктура автоматизации	8
2.3. Тестирование	18
3. Выводы	18
4. Список использованных источников	19

1. Задача

В рамках лабораторной работы требуется изучить основные положения темпоральной логики LTL и получить практические навыки формализации требований с использованием темпоральных выражений. Для заданных условий необходимо составить соответствующие LTL-формулы и обосновать их структуру и смысл.

Для проверки корректности составленных формул требуется изучить работу верификатора SPIN и языка Promela, а также подготовить набор тестовых моделей, включающих как сценарии, удовлетворяющие проверяемым формулам, так и сценарии, нарушающие заданные условия. При разработке тестов необходимо учитывать различные варианты начальных значений атомарных предикатов.

Дополнительно требуется разработать инфраструктуру автоматизации, позволяющую задавать каждую проверяемую формулу в одном месте, запускать проверку всех подготовленных тестовых примеров одной командой и автоматически формировать результаты выполнения тестов.

2. Ход работы

2.1. LTL формулы

В рамках задания требуется решить ряд заранее сформулированных задач на построение LTL-формул, а также самостоятельно придумать выражения и выполнить их формализацию.

Первая группа заданий посвящена формализации утверждений с одним атомарным предикатом p , описывающим событие «я выхожу замуж».

1.1. Задача: «Я выйду замуж не менее двух раз».

Решение: $\langle \rangle (p \ \&\& \ X \ \langle \rangle \ p)$

Описание: Формула утверждает, что в некоторый момент событие произойдет, а после этого, начиная со следующего момента времени, оно

обязательно произойдет еще раз. Таким образом, гарантируется как минимум два отдельных наступления события, при этом их общее количество не ограничивается.

1.2. Задача: «Я выйду замуж не более двух раз».

Решение: $\Box(p \rightarrow X \Box(p \rightarrow X \Box !p))$

Описание: Формула допускает первое наступление события, после которого допускается еще одно наступление. Начиная с момента после второго события, предикат должен оставаться ложным всегда. Таким образом, событие может произойти ноль, один или два раза, но не более двух.

1.3. Задача: «Я выйду замуж точно два раза».

Решение: $!p \cup (p \&\& X (!p \cup (p \&\& X \Box !p)))$

Описание: Формула задает последовательность: сначала событие отсутствует, затем происходит первый раз, после него снова отсутствует, происходит второй раз, а далее больше не наступает. При этом оператор $\cup$ не требует, чтобы его левая часть выполнялась хотя бы один момент времени: правая часть может стать истинной сразу. Поэтому промежутки отсутствия события могут иметь нулевую длину, и, например, последовательность 1100... также удовлетворяет формуле.

1.4. Задача: «Я выйду замуж не менее одного раза».

Решение: $\langle \rangle q$

Описание: Формула требует, чтобы событие произошло хотя бы один раз в некоторый момент времени. При этом не накладывается никаких ограничений на количество последующих наступлений события.

1.5. Задача: «Я выйду замуж не более одного раза».

Решение: $\Box(p \rightarrow X \Box !p)$

Описание: Формула допускает отсутствие события или его однократное наступление. Если событие произошло, то начиная со следующего момента

времени предикат должен всегда оставаться ложным. Таким образом, повторное наступление события исключается.

1.6. Задача: «Я выйду замуж точно один раз».

Решение: $\neg p \vee (p \wedge X \neg p)$

Описание: Формула задает, что до наступления события предикат остается ложным, затем событие происходит один раз, а начиная со следующего момента времени больше никогда не наступает. При этом оператор $\vee$ допускает, что правая часть станет истинной сразу, поэтому событие может произойти уже в начальный момент времени.

1.7. Задача: «Я никогда не выйду замуж».

Решение: $\neg p$

Описание: Формула требует, чтобы предикат оставался ложным во все моменты времени. Таким образом, событие не может произойти ни в начальный момент, ни в любой последующий.

Далее идет группа заданий на два атомарных предиката. У них, в отличие от первой части, нет какого-то особого смысла.

2.1. Задача: «Событие а может случиться только во время b».

Решение: $\neg(a \rightarrow b)$

Описание: Формула требует, чтобы в каждый момент времени наступление события а было возможно только при одновременной истинности b. При этом само событие а не обязано происходить, а b может быть истинно независимо от а.

2.2. Задача: «Событие а обязательно случится, но только во время b».

Решение: $(\rightarrow a) \wedge \neg(a \rightarrow b)$

Описание: Формула состоит из двух условий: событие а должно обязательно произойти хотя бы один раз, и при каждом его наступлении предикат

b должен быть истинным в тот же момент времени. При этом b может быть истинным и независимо от события a.

2.3. Задача: «События a и b обязательно произойдут хотя бы один раз, при этом событие a может произойти только строго после b».

Решение: $(!a \cup (b \ \&\& \ !a)) \ \&\& \ \<\> \ a$

Описание: Формула состоит из двух условий. Оператор $\cup$ требует, чтобы до наступления b предикат a оставался ложным, причём в момент наступления b он также должен быть ложным. Вторая часть формулы с оператором $\<\>$ гарантирует, что событие a обязательно произойдёт хотя бы один раз. Таким образом, гарантируется наступление обоих событий, причём первое наступление a возможно только строго после b. При этом количество последующих наступлений событий и промежуток времени между ними не ограничиваются.

Далее необходимо придумать самостоятельно пять заданий на один атомарный предикат.

3.1. Задача: «После каждого наступления события a в следующий момент событие a обязательно должно быть ложно».

Решение: $\square(a \rightarrow X \ !a)$

Описание: Формула требует, чтобы после каждого момента, в котором событие a происходит, в следующий момент предикат a обязательно был ложным. Таким образом, два наступления события a не могут происходить в соседние моменты времени.

3.2. Задача: «Когда-нибудь должна встретиться последовательность: a, затем не a, затем снова a».

Решение: $\<\> \ (a \ \&\& \ X \ (!a \ \&\& \ X \ a))$

Описание: Формула требует, чтобы в некоторый момент встретилась последовательность из трех состояний: сначала a истинно, в следующий момент

а ложно, а еще через один момент а снова истинно. Остальные моменты времени формула не ограничивает.

3.3. Задача: «С некоторого момента значение события а должно строго чередоваться».

Решение: $\langle \rangle []((a \rightarrow X!a) \&\& (!a \rightarrow Xa))$

Описание: Оператор $\langle \rangle$ позволяет выбрать некоторый момент времени, начиная с которого вступает в силу условие внутри $[]$. Оператор $[]$ требует выполнять это условие на всех последующих шагах. Две импликации совместно задают противоположное значение а на следующем шаге независимо от его текущего значения, поэтому после выбранного момента возникает бесконечное чередование а, !а, а, !а,

3.4. Задача: «Событие а произойдет хотя бы один раз, но с некоторого момента больше никогда происходить не будет».

Решение: $(\langle \rangle a) \&\& (\langle \rangle []!a)$

Описание: Первая часть формулы с оператором $\langle \rangle$ гарантирует существование хотя бы одного момента, в котором а истинно. Вторая часть требует существования некоторого момента, после которого условие !а выполняется всегда. Совместно эти части задают хотя бы одно наступление события до момента его окончательного прекращения.

3.5. Задача: «С некоторого момента значение а стабилизируется и больше не будет изменяться».

Решение: $\langle \rangle ([]a \parallel []!a)$

Описание: Оператор $\langle \rangle$ допускает произвольное поведение а до некоторого момента времени. В найденный момент должно выполниться одно из двух условий: либо а остается истинным во всех последующих состояниях, либо ложным. Дизъюнкция позволяет зафиксироваться любому из двух возможных значений.

В последней части задания на LTL формулы необходимо придумать самостоятельно пять заданий на два атомарных предиката.

4.1. Задача: «Каждый раз, когда a и b одновременно истинны, в следующий момент оба должны быть ложны».

Решение: $\square((a \ \&\& \ b) \rightarrow X (\neg a \ \&\& \ \neg b))$

Описание: Импликация проверяется на каждом шаге: если в текущий момент одновременно истинны a и b , оператор X переносит требование на следующий момент времени, где оба предиката должны быть ложными. Если хотя бы один из предикатов в текущий момент ложен, правая часть импликации не накладывает дополнительных ограничений.

4.2. Задача: «После каждого a предикат b должен быть истинным до следующего a , причём следующее a обязательно должно наступить. Если a идут подряд, b между ними не требуется».

Решение: $\square(a \rightarrow X(b \ U \ a))$

Описание: Импликация проверяется после каждого наступления a . Оператор X переносит проверку на следующий момент времени, после чего b должно оставаться истинным вплоть до следующего наступления a . Использование сильного оператора U дополнительно гарантирует, что это следующее a действительно наступит.

4.3. Задача: «Когда-нибудь должна встретиться последовательность: только a , затем только b , затем снова только a ».

Решение: $\langle \rangle ((a \ \&\& \ \neg b) \ \&\& \ X((b \ \&\& \ \neg a) \ \&\& \ X(a \ \&\& \ \neg b)))$

Описание: Оператор $\langle \rangle$ требует, чтобы указанная комбинация состояний встретила хотя бы один раз. Внутри неё с помощью двух операторов X последовательно задаются три соседних момента времени: сначала истинно только a , затем только b , а затем снова только a . Остальное поведение предикатов формулой не ограничивается.

4.4. Задача: «С некоторого момента события a и b всегда должны иметь противоположные значения».

Решение: $\langle \rangle []((a \ \&\& \ !b) \ || \ (!a \ \&\& \ b))$

Описание: Оператор $\langle \rangle$ задаёт момент, начиная с которого условие должно выполняться постоянно, а оператор $[]$ фиксирует это требование для всех последующих состояний. Внутренняя дизъюнкция допускает только два варианта: a истинно при ложном b или b истинно при ложном a , тем самым исключая их одновременную истинность или ложность.

4.5. Задача: «После каждой пары последовательных моментов, в которых a истинно, в непосредственно следующий момент b должно быть истинным».

Решение: $[]((a \ \&\& \ X \ a) \ -> \ X \ X \ b)$

Описание: Формула проверяется в каждый момент времени. Если a истинно сейчас и остаётся истинным на следующем шаге, то двойной оператор X требует истинности b через два шага относительно текущего момента. Иными словами, после пары последовательных состояний с a равным истинному значению b обязательно должно стать истинным на следующем после них шаге.

2.2. Инфраструктура автоматизации

Далее в рамках задания необходимо разработать «инфраструктуру автоматизации», которая позволит проверить правильность написанных ранее формул на тестах. Важнейшим требованием к такой системе является гибкость: она должна позволять выполнять автоматические тесты с разными начальными состояниями атомарного предиката, а также тестировать формулу как на «правильных» кейсах, так и на «неправильных», чтоб показать, что формула работает корректно в разных ситуациях.

К разрабатываемой системе предъявляются следующие требования:

1. Единое описание задания. Каждая LTL-формула должна задаваться только один раз вместе с используемым набором атомарных предикатов и поддерживать несколько независимых тестов.

2. Гибкое описание сценариев. Каждый тест должен позволять задавать начальные значения всех предикатов, конечную последовательность состояний и бесконечно повторяющийся цикл.

3. Проверка ожидаемого результата. Для каждого теста должно указываться, ожидается выполнение или нарушение формулы. Тест считается успешным, если результат проверки совпадает с ожиданием.

4. Расширяемость. Добавление новых формул и тестов должно выполняться через файлы данных, без изменения кода системы и ручного создания моделей Promela.

5. Автоматизация и отчётность. Проверка всех тестов должна запускаться одной командой и завершаться формированием текстового отчёта с перечнем тестов и статусами их прохождения.

Для выполнения этих требований выбрана структура JSON: на уровне задания хранятся формула, список предикатов и набор тестов, а внутри каждого теста последовательность состояний, необязательный цикл и ожидаемый результат:

```
{
  "title": "<описание проверяемого свойства>",
  "predicates": ["a", "b"], // Список предикатов
  "formula": "<LTL-формула с предикатами a и b>",
  "tests": [
    {
      "name": "<уникальное имя теста>",
      "note": "<описание сценария и обоснование ожидания>",
      "prefix": [
        {"a": 0, "b": 1},
        {"a": 1, "b": 0}
      ],
      "loop": [
        {"a": 0, "b": 0},
        {"a": 1, "b": 1}
      ],
      "expected": true
    }
  ]
}
```

```
}
```

Массив `tests` может содержать несколько тестов для одной формулы. В каждом тесте `prefix` задаёт конечную последовательность состояний, начиная с начального, а `loop` бесконечно повторяющееся продолжение. Если `loop` отсутствует, после `prefix` все предикаты становятся ложными и сохраняют это значение. Поле `expected` задаёт ожидаемый результат: `true` — формула выполняется, `false` — нарушается.

«Инфраструктура автоматизации» представляет собой скрипт на `python`, который заполняет шаблон на языке `Promela`, запускает сборку и проверку его, используя консольный верификатор `SPIN` и компилятор `gcc`, после чего дожидается завершения выполнения и сравнивает результат выполнения с ожидаемым.

Для работы приложения необходимо добавить в переменную `path` пути к `gcc` и `spin`, либо записать эти переменные в конфиг:

```
{
  "spin": "%LOCALAPPDATA%/Programs/Spin/spin.exe",
  "cc": "C:/msys64/ucrt64/bin/gcc.exe"
}
```

Сама программа представляет собой скрипт на языке `python`, весь код которой можно найти по ссылке [1].

Далее будут коротко рассмотрены ключевые этапы работы скрипта.

```
def main():
    # Парсим консоль
    parser = argparse.ArgumentParser(description=__doc__,
    formatter_class=argparse.RawDescriptionHelpFormatter)
    parser.add_argument("--tasks", type=Path, default=ROOT / "tasks",
    help="Папка JSON-файлов")
    parser.add_argument("--test", default="", help="Фильтр задание/тест,
    допускает *")
    parser.add_argument("--timeout", type=float, default=30, help="Лимит
    секунд на одну команду")
    parser.add_argument("--depth", type=int, default=10000, help="Максимальная
    глубина поиска")
    parser.add_argument("--replay", type=Path, help="Повторить контрпример из
    папки ранее выполненного теста")
    args = parser.parse_args()
    if args.timeout <= 0 or args.depth <= 0:
        parser.error("--timeout и --depth должны быть положительными")
```

Программа представляет из себя консольную утилиту, у которой есть несколько полезных параметров, а именно:

`--tasks` — позволяет задать папку, в которой указаны json-файлы с тестами.

`--test` — позволяет задавать какие именно задания и тесты из этих заданий запустить в рамках вызова.

`--timeout`, `--depth` — параметры для вызова SPIN, которые позволяют изменять глубину поиска и ограничение по времени ожидания ответа от процесса.

`--replay` — полезный параметр, который позволяет посмотреть контрпример из ранее выполненного теста. Для этого указывает путь к тесту, в котором был найден контрпример.

Далее происходит загрузка данных из конфига, а также обновление PATH для вызова gcc и SPIN (с кодом можно ознакомиться в [1]).

После обработки всей этой служебной информации у программы есть 2 варианта выполнения: воспроизведение replay или основная программа работы. Начнем со второй, в коде оставлены комментарии, которые позволяют подробно ознакомиться с происходящим:

```
build = ROOT / "build"
build.mkdir(exist_ok=True)
run_folder = Path(tempfile.mkdtemp(
    prefix=datetime.now().strftime("%Y%m%d_%H%M%S_"), dir=build)
)
report = run_folder / "report.txt"
results, problem, environment = [], "", ""
try:
    # Загрузили задания
    tasks = load_tasks(args.tasks)
    # Поиск выбранных тестов по шаблону из args.test
    selected = [
        (task, test.copy()) for task in tasks for test in task["tests"]
        if fnmatch.fnmatchcase(f"{task['id']}/{test['name']}", args.test)
    ]
    if not selected:
        raise ValueError(f"Нет тестов по фильтру {args.test!r}")
    # Загрузка шаблона
    template = (ROOT / "template.pml").read_text(encoding="utf-8")
```

```

# Проверка SPIN
version = run_command([spin, "-V"], run_folder, "environment.log",
                      args.timeout)
if version.returncode != 0:
    raise RuntimeError("SPIN не запускается. Проверьте config.json")
environment = version.stdout.strip()
# Проверяем возможность перевода X
probe = run_command([spin, "-f", "X probe"], run_folder,
                    "next_support.log", args.timeout)
if probe.returncode != 0:
    raise RuntimeError("SPIN не поддерживает X; нужна сборка с -DNXT.")
# Проходим по каждому таску
for task, test in selected:
    # Запуск теста
    result = run_test(task, test, template, run_folder / task["id"] /
                     test["name"], spin, cc, args.timeout, args.depth)
    results.append(result)
    # Вывод результата теста
    print(f"[{result['status']}] {task['id']}/{test['name']}:
          {result['detail']}", flush=True)
    if result["status"] == "FAIL" and result["trace"]:
        print(result["trace"], flush=True)
except (ValueError, OSError, RuntimeError) as error:
    problem = str(error)
    print(f"ERROR: {problem}", file=sys.stderr)
# Запись общего репорта
summary = write_report(results, report, run_folder, environment, problem)
# Вывод итогов
print(summary)
print(f"Отчёт: {report.resolve()}")
if problem or any(r["status"] == "ERROR" for r in results):
    return 2
return 1 if any(r["status"] == "FAIL" for r in results) else 0

```

В данном фрагменте создается отдельная папка для артефактов текущего запуска и загружается список заданий. После этого выбираются тесты, соответствующие указанному шаблону, и загружается шаблон Promela-модели. Далее выполняется проверка доступности SPIN и поддержки оператора X.

После подготовки окружения код последовательно запускает выбранные тесты, сохраняет их результаты и при необходимости выводит трассировку ошибок. По завершении формируется общий текстовый отчет с результатами проверки, а программа возвращает код завершения в зависимости от наличия ошибок или неуспешных тестов.

Здесь используется несколько функций. `load_tasks` читает json файлы и проверяет их на корректность, после чего возвращает все найденные. Код можно найти в [1].

Следующая функция `run_command`:

```
def run_command(command, folder, log_name, timeout):
    # Преобразование листа в команду
    command_text = subprocess.list2cmdline(command) if os.name == "nt"
                                                else shlex.join(command)

    heading = f"$ {command_text}\n\n"
    try:
        # Запуск команды, весь вывод в result
        result = subprocess.run(
            command, cwd=folder, stdout=subprocess.PIPE,
            stderr=subprocess.STDOUT,
            text=True, encoding="utf-8", errors="replace", timeout=timeout,
        )
    except subprocess.TimeoutExpired as error: # Ошибка по времени
        output = error.stdout or b""
        if isinstance(output, bytes):
            output = output.decode("utf-8", errors="replace")
        # Записываем частичный вывод
        (folder / log_name).write_text(heading + output + "\nTIMEOUT\n",
                                       encoding="utf-8")

        # Запуск ошибки
        raise RuntimeError(f"Тайм-аут {timeout} с; см. {log_name}") from error
    except OSError as error: # Невозможность запуска (например нет прав)
        (folder / log_name).write_text(heading + str(error), encoding="utf-8")
        raise RuntimeError(f"Не удалось запустить {command[0]}; см.
                           {log_name}") from error

    # Сохраняем вывод
    (folder / log_name).write_text(heading + result.stdout, encoding="utf-8")
    return result
```

Как понятно из названия она необходима исключительно как обертка для запуска команд в консоли, с обработкой различных ошибок, которые могут возникать в процессе.

Вторая функция, которая используется в основном коде это `run_test`:

```
def run_test(task, test, template, folder, spin, cc, timeout, depth):
    """Сгенерировать, проверить, сравнить с ожиданием"""
    folder.mkdir(parents=True)
    # Создаем модель по шаблону
    (folder / "model.pml").write_text(generate_model(task, test, template),
                                       encoding="utf-8")

    # Копирование конкретного теста
    (folder / "case.json").write_text(json.dumps(
        {"task": {k: task[k] for k in ("id", "formula", "predicates",
                                       "source")}, "test": test},
        ensure_ascii=False, indent=2) + "\n", encoding="utf-8")

    # Создание заготовки результата
    result = {"task": task, "test": test, "folder": folder,
              "status": "ERROR", "actual": None, "detail": "", "trace": ""}

    # Обработка теста
    try:
        executable = "pan.exe" if os.name == "nt" else "pan"
        # Генерация и компиляция
        commands = [
            ([spin, "-o3", "-a", "model.pml"], "generate.log"),
            ([cc, "-O2", "-DNOREDUCE", "-o", executable, "pan.c"],
             "compile.log"),
```

```

1
for command, log in commands:
    process = run_command(command, folder, log, timeout)
    if process.returncode != 0:
        raise RuntimeError(f"Ошибка генерации или компиляции; см.
                           {log}")

# Запуск обработки
process = run_command(
    [str(folder / executable), "-a", "-b", f"-m{depth}", "-w18"],
    folder, "verify.log", timeout,
)
actual = parse_result(process.stdout, process.returncode, (folder /
    "model.pml.trail").exists())
result["actual"] = actual
result["status"] = "PASS" if actual == test["expected"] else "FAIL"
result["detail"] = "Формула выполняется" if actual else "Найден
                    контрпример"

if not actual:
    result["trace"] = replay_trace(spin, folder, timeout)
except RuntimeError as error:
    result["status"] = "ERROR"
    result["detail"] = str(error)
return result

```

Функция создает отдельную папку для конкретного теста, генерирует Promela-модель по шаблону и сохраняет описание проверяемого случая. Далее SPIN генерирует код верификатора, после чего он компилируется и запускается с заданной глубиной поиска.

Полученный результат сравнивается с ожидаемым значением теста. В зависимости от совпадения устанавливается статус PASS или FAIL. Если найден контрпример, дополнительно восстанавливается и сохраняется трассировка его выполнения. Ошибки генерации, компиляции или запуска фиксируются со статусом ERROR.

Здесь есть уже знакомые вызовы `run_command`, функция `replay_trace` будет рассмотрена чуть позже так как она же вызывается при флаге `--replay`, функция `parse_result` является лишь набором `if`ов, которая проверяет, что вернул SPIN, проверяя найден контрпример или нет, или же возникла какая-то другая ошибка во время запуска, её код можно найти в [1]. Самой важной функцией является `generate_model`, она берет модель и заполняет её. Для реализации используется класс `Template` из библиотеки `string`: создается файл «шаблон» с метками, которые можно заполнить в рантайме.

Шаблон выглядит следующим образом:

```
$declarations

init {
$body
}

ltl check { $formula }
```

Здесь есть блок `declarations`, в котором задаются начальные состояния предикатов, потом `body`, в котором заполняется сам тест и блок `formula`, который заполняется LTL формулой.

Функция, которая заполняет шаблон приведена ниже:

```
def generate_model(task, test, template):
    """Один элемент последовательности = один d_step, включая вывод для отладки."""
    predicates = task["predicates"]
    initial = test["prefix"][0]
    # Начальные состояния
    declarations = "\n".join(f"bool {p} = {int(initial[p])};"
                             for p in predicates)

    def step(state, label):
        # Задаёт нужные присваивания
        assignments = " ".join(f"{p} = {int(state[p])};" for p in predicates)
        # Предикаты для вывода
        values = ", ".join(f"{p}=%d" for p in predicates)
        # printf внутри d_step не добавляет отдельного момента времени.
        trace =
            f'printf("TRACE {label}: {values}\\n", {", ".join(predicates)});'
        # Итоговый шаг состоит из присваиваний и вывода
        return f"d_step {{ {assignments} {trace} }}"

    # Основное тело, последовательно проходит по каждому префиксу
    body = [" " + step(state, f"prefix[{i}]"
                     for i, state in enumerate(test["prefix"][1:], start=1)]
    # Обработка loop, если нету, все предикаты = 0
    loop = test.get("loop", [{p: 0 for p in predicates}])
    body.append(" do")
    for i, state in enumerate(loop):
        # Генерирует подпись
        label = f"loop[{i}]" if "loop" in test else "default_zero"
        # Генерация шага цикла
        body.append((" " if i == 0 else " ") + step(state, label))
    body.append(" od;")
    # Запись в шаблон
    return Template(template).substitute(
        declarations=declarations, body="\n".join(body),
        formula=task["formula"]
    )
```

Функция формирует текст Promela-модели для конкретного теста. Сначала из задания берется список предикатов, а из первого элемента `prefix`

их начальные значения. На их основе создаются объявления переменных типа `bool`. Первый элемент `prefix` задается при объявлении переменных, поэтому в основное тело модели добавляются только последующие элементы последовательности.

Для формирования каждого шага используется вложенная функция `step`. Она создает присваивания значений всем предикатам и команду `printf`, которая выводит метку шага и текущее состояние переменных. Присваивания и вывод объединяются в блок `d_step`, поэтому один элемент последовательности обрабатывается как один неделимый шаг, а отладочный вывод не создает дополнительного момента времени.

После прохождения `prefix` модель переходит к бесконечному повторению последовательности `loop`. Для этого создается конструкция `do ... od` с одной ветвью, внутри которой последовательно выполняются все элементы цикла. Если `loop` не указан, вместо него используется одно состояние, в котором все предикаты равны нулю. Таким образом, после завершения префикса модель продолжает выполнение с нулевыми значениями предикатов.

В конце сформированные объявления, тело модели и LTL-формула подставляются в соответствующие метки шаблона с помощью `Template.substitute`. Функция возвращает готовый текст модели, который затем сохраняется в файл `model.pml`.

Вот пример такого сгенерированного файла:

```
bool a = 0;
bool b = 0;

init {
  d_step { a = 0; b = 1; printf("TRACE prefix[1]: a=%d, b=%d\n", a, b); }
  d_step { a = 0; b = 1; printf("TRACE prefix[2]: a=%d, b=%d\n", a, b); }
  d_step { a = 1; b = 0; printf("TRACE prefix[3]: a=%d, b=%d\n", a, b); }
  do
    :: d_step { a = 0; b = 0; printf("TRACE default_zero: a=%d, b=%d\n", a,
b); }
  od;
```

```
}  
ltl check { <> (b && X (b U a)) }
```

К этому моменту были рассмотрены все функции, кроме `replay_trace`, она используется тогда, когда в результате выполнения теста находится контрпример или при вызове флага `--replay`, чтоб вывести контрпример в консоль.

```
def replay_trace(spin, folder, timeout):  
    """Воспроизвести настоящий .trail и выделить из него состояния  
    сценария."""  
    # Загрузка описания задачи и тестов  
    snapshot = json.loads((folder / "case.json").read_text(encoding="utf-8"))  
    task, test = snapshot["task"], snapshot["test"]  
    # Запуск команды  
    log_path = (folder / "replay.log").resolve()  
    result = run_command([spin, "-t", "-p", "-g", "model.pml"], folder,  
                        log_path, timeout)  
    if result.returncode != 0 or "cannot find trail file" in result.stdout:  
        raise RuntimeError(f"Не удалось воспроизвести контрпример; см.  
                           {log_path}")  
    # Обработка  
    # Выводим первую строку (её нет в .trail)  
    lines = ["Контрпример из SPIN .trail:",  
            f"t=0: {state_text(test['prefix'][0], task['predicates'])}  
            (prefix[0])"]  
    tick = 0  
    # Выводим все строки  
    for line in result.stdout.splitlines():  
        if "START OF CYCLE" in line:  
            lines.append("    --- начало повторяемого цикла SPIN ---")  
            match = re.match(r"\s*TRACE ([^:]+): (.*)", line)  
            if match:  
                tick += 1  
                lines.append(f"t={tick}: {match.group(2)} ({match.group(1)})")  
    lines.append(f"Полные шаги модели и автомата LTL: {log_path}")  
    trace = "\n".join(lines)  
    (folder / "trace.txt").write_text(trace + "\n", encoding="utf-8")  
    return trace
```

Сначала загружаются данные этого теста, после чего SPIN воспроизводит последовательность действий, при которой было обнаружено нарушение.

Полученный вывод обрабатывается, чтобы представить контрпример в виде понятной последовательности состояний предикатов. Начальное состояние добавляется отдельно, поскольку оно задается при объявлении переменных модели. Далее выбираются состояния, выводимые при выполнении шагов сценария, и каждому из них присваивается

последовательный номер времени. Если в трассировке встречается начало повторяемого цикла, оно отмечается отдельно: это позволяет увидеть, какая часть выполнения будет повторяться бесконечно.

Таким образом, пользователь может проследить изменение предикатов и понять причину нарушения формулы. Сформированная трассировка сохраняется и возвращается для включения в отчет или вывода в консоль. Если воспроизвести контрпример не удалось, функция сообщает об ошибке.

2.3. Тестирование

Тестовые примеры для каждой задачи создавались вручную. Перебирались все возможные начальные состояния, тестировались различные ожидаемые результаты выполнения. Важно было не подгонять тесты под формулу, поэтому тесты писались исключительно по текстовой формулировке задания. Также обязательно тестировалось поведение формул при различных вариантах последовательностей на бесконечности.

В результате получилось 20 файлов тестов (по одному для каждой формулы), все тесты выполнились успешно:

```
PS C:> python run.py
[PASS] 1.1 marriage_twice/never: Найден контрпример
...
[PASS] 4.5 consecutive_a_then_b/always_a_without_b: Найден контрпример
Итого: 255 тестов; PASS=255, FAIL=0, ERROR=0.
Отчёт: C:\build\20260916_011909_5jwypc88\report.txt
```

Подробнее с каждым тестовым случаем можно ознакомиться по ссылке [1], а также выполнить запуск самостоятельно.

3. Выводы

В рамках лабораторной работы были изучены основные принципы темпоральной логики LTL и получены практические навыки формализации требований с помощью темпоральных выражений. Для заданных и самостоятельно составленных условий были построены LTL-формулы и

рассмотрен смысл используемых операторов. Для проверки формул были изучены язык Promela и верификатор SPIN.

Также была разработана инфраструктура автоматизации, позволяющая хранить формулы и тестовые сценарии отдельно от программного кода, автоматически генерировать Promela-модели, выполнять их проверку и формировать отчет о результатах. Всего было подготовлено 20 наборов тестов, включающих различные начальные состояния и варианты бесконечного поведения. Все 255 тестовых случая завершились успешно, что подтвердило соответствие составленных формул исходным условиям задач.

4. Список использованных источников

1. DafterT. LTL_checker: репозиторий исходного кода [Электронный ресурс].
- GitHub. - URL: https://github.com/DafterT/LTL_checker.git (дата обращения: 17.09.2026).